

Assessment Pass Task 1

Jordan Elkin

Introduction

Problem Statement

Living in a busy house with multiple people, rooms such as the bathroom (in this context a bathroom is defined as a space with a toilet, basin and shower in one area) can always seem to be occupied. This project aims to find out when the primary bathroom is free and for how long it is.

Use Case

Finding out how long the bathroom is in use will enable people living in the house to know when they can make use of the bathroom. Given that in this case the bathroom is both a toilet, and showering area as well as a preparation area. It is vital information for anyone in a shared household to know when they can have their turn of the space. Further, to this the information provided by this project will enable people living within the household to be more mindful of their use of the bathroom area and ensure that the space is equally shared. The information collected will also assist in giving residents and understanding of when most people are attempting to use the space and when the best time for them to use the space will be on average.

Expected Outcomes

The pattern and insights that are expected to be found by this project are how often the space is being utilised by people within the household (i.e. how long someone is spending inside the space and also how often people are visiting the space as well). Other patterns that are expected to be found is when are the peak times which people are making use of the space as well as the average time difference between the space being free and in use. This information will inform residents the best time to visit the area and also provide insights as to how everyone is using the space and if changes should be made.

Sensor Selection

The that have been selected to monitor and identify when people are utilising the bathroom are the passive infrared (PIR) motion sensor (HC-SR501), DHT11 temperature and humidity module, and a photo-resistor. The motion sensor has been selected for use in the project as this sensor will allow for identifying when someone is inside of the space. The DHT11 temperature and humidity sensor will enable the arduino to identify presence as well and verify the data that the PIR has detected. But also enable the identification of shower usage. This is because typically as the shower is used the relative humidity in the room will increase and same with the temperature. This means we are able to collect de-identified data without impacting people's personal privacy. Lastly, the photo-resistor was selected as this will enable tracking of light usage during. This is relevant to some houses where the bathroom area is closed off and has no natural light. However, day-light can be accounted for by enabling the user to specify environment conditions before the data is recorded.

Sensor Specifications:

Sensor Specification Overview

Sensor	Accuracy	Measurement Ranges	Sampling Rate (Hz)	Output Format
DHT11	±5% Relative Humidity ±2 Degrees celsius	20-90% Relative Humidity 0-50 Degrees celsius	1	2 bytes for Humidity 2 bytes for Temperature (8 bits for integral reading 8 bits for decimal point representation).
PIR Motion Sensor	Non-precision sensor detects motion and drops voltage to low on data pin	3-7 m within a 120 degree arc	0.17	Sets data pin from high to low for detected motion
Photo-resistor	±43% at 10 lux	1-100 lux	20	Pin becomes high as resistance lessens due to increase in light

All sensor specifications are directly from the relevant data-sheet for the sensor. Please see the references section for the specific publications.

DHT11 Humidity and Temperature Sensor

The DHT11 humidity and temperature sensor is a simple, accurate and inexpensive sensor. This sensor is able to capture both the relative humidity and temperature of its environment. The sensor is accurate to within plus or minus 5% for the measurement of relative humidity and within plus or minus two degrees celsius for temperature measurements. The DHT11 is able to output a sample at a rate of 1 sample every second or 1 Hz. This data is sent over the signal wire as a stream of 40 bits with the first 16 bits containing the integral (non-decimal) whole number reading for relative humidity and also the numbers which make up the decimal component. The next 16 bits are representative of the temperature reading and the last 8 bits are the checksum for ensuring all of the data was correctly received. The DHT11 is able to measure relative humidity between 20-90 % humidity and temperature between 0-50 degrees celsius. This makes the DHT11 an integral sensor for use within this project. This is because this sensor will be able to identify when the area is occupied and can lead to inferences about how the bathroom space was utilised. For example making inferences about using the shower.

HC-SR501 Passive Infrared Motion Sensor

The HC-SR501 Passive Infrared Motion Sensor (PIR Sensor) is a sensor capable of detecting motion within 3 – 7 metres within a 120 degree arc. This sensor is able to output a detection every 6 seconds using the default settings. However, can be adjusted output a detection signal every 3 seconds using the repeatable trigger function. This means that the sampling rate of this sensor is approximately 0.17 Hz making it not possible to collect a sample until after the 6 second window. Therefore, any movement that occurs within the 6 second restart window will not be counted such as quick change over of people moving in and out of the room. Moreover, the PIR sensor is able to operate at a minimum of 5v making it easy to incorporate into the project. Further, this sensor works by pulling the pin that it is connected to on the micro-controller down to register motion. This means that a custom format will be created in the form of timestamp, motion status. Where the status of the motion can be either true or false.

Photo-resistor

The GL5506 Series photo resistor is a type of photocell that is able to detect light by reducing resistance as the amount of light on the sensor decreases. This means that power will flow through the cell once enough light has been reduced (This can naturally be swapped if the circuit is changed where the opposite is true that the more light detected the higher the voltage will be). The reaction to the rise in light is approximately 45 ms and the reaction time to the fall in the presence of light is approximately 55 ms. This results in an approximate average sampling rate of 20 Hz which ensures there is enough time for either event to occur before the micro-controller reads the status of the pin. The output for a photo resistor is an analog

reading of the voltage which can be used to determine the luminosity for the environment.

Data Capture Setup

The sensors will be directly connected to the Arduino Nano through the boards GPIO pins. This will allow for a direct interface with the sensors and allow for the data to be sent and received over the GPIO pins. To ensure all the sensors receive power the main power circuit will run through a breakout board (bread board) where the sensors will have their power and ground pins connected to. For the photo-resistor a voltage divider circuit will be created with a 10k ohm resistor connected to ground and the negative side of the resistor. Further, the 'sensing' wire will be in between the photo-resistor and the fixed resistor. This will ensure that the data will be captured from the resistor so that when it is bright the voltage will be positive and dark will have a lower voltage. For communication between the arduino and the laptop computer the primary means will be over the serial connection provided by the micro-USB port on the arduino. This will enable the data that has been captured by the arduino to be collected in a python program where further analysis and tagging will occur.

Due to the various devices having different sampling speeds and startup requirements an initial 2 minute wait time will occur before sampling will begin. This is because the PIR sensor requires that amount of time as a minimum to initialise its own IC and ensure the readings are accurate. Further, the sampling time will be slightly different for each of the different sensors. For the PIR sensor sampling will occur every 6 seconds or 0.17 Hz so that the data collected will be accurate. The device will also be setup for non-repeatable mode so that anyone inside of the arc will be detected until they leave the room. The sampling rates for the DHT11 and the photo-resistor will occur at a rate of 20 Hz. Since the photocell will have the most samples available and at the greatest speed, most samples will be redundant. Therefore, slowing the rate to 1 Hz and sampling at the same rate as the DHT11 is more beneficial as the data will show more of a change per each sample.

The intended duration for real world capture should be for at least a minimum of a couple of days up to a week. This would enable a more accurate picture from the sensors on how an area such as a bathroom is utilised and whether the sensors are able to truly detect how the room is being utilised and for how long. However, this is not currently feasible therefore a controlled simulation for approximately 30 minutes should at least provide enough data for a proof of concept. This means that after 30 minutes enough data should be available to display average time occupied, amount of visits to the space, shower usage inferred by the change in relative humidity and temperature. The arduino upon receiving the data from the sensors will send the information over serial to the laptop in the format `sensor_name,data`. Since this process will take less than a second the laptop will be able to append the current date time to the sensor data. All of the collected data will be stored in a python dictionary

(hash table) which will then be written out to a CSV file for further processing and storage.

Local Processing and Storage

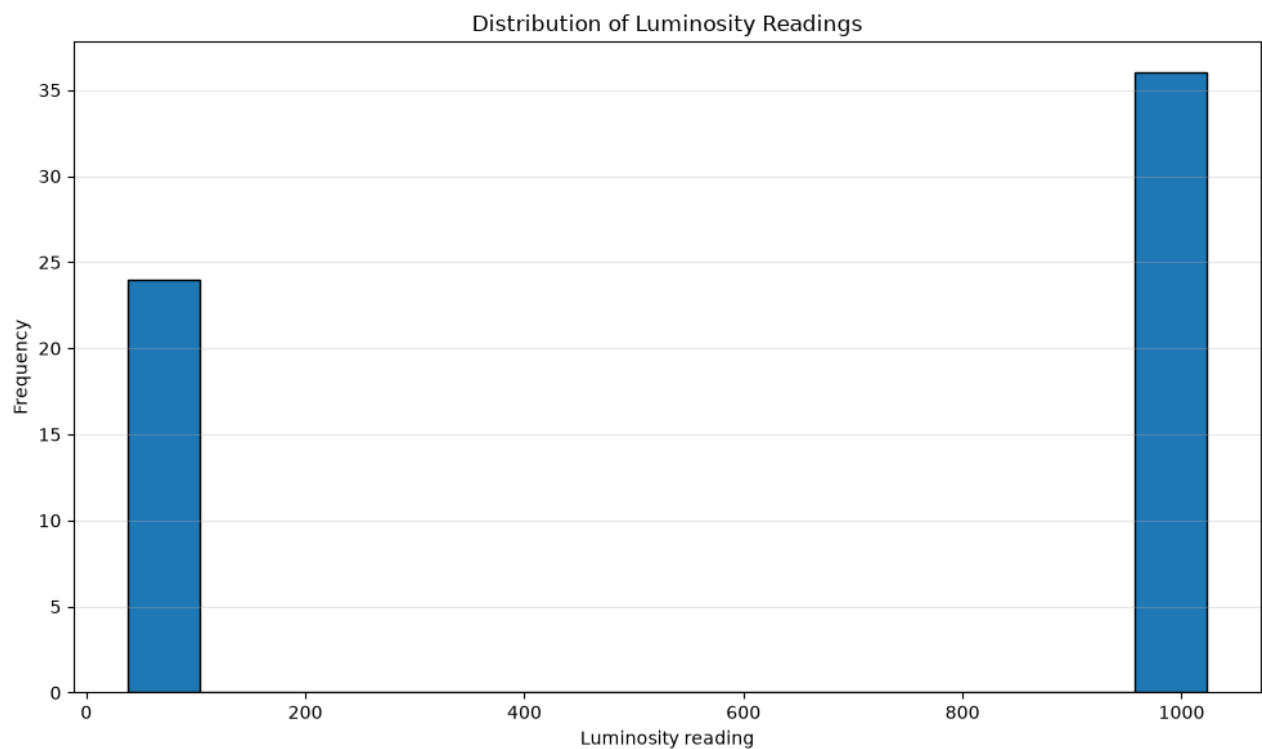
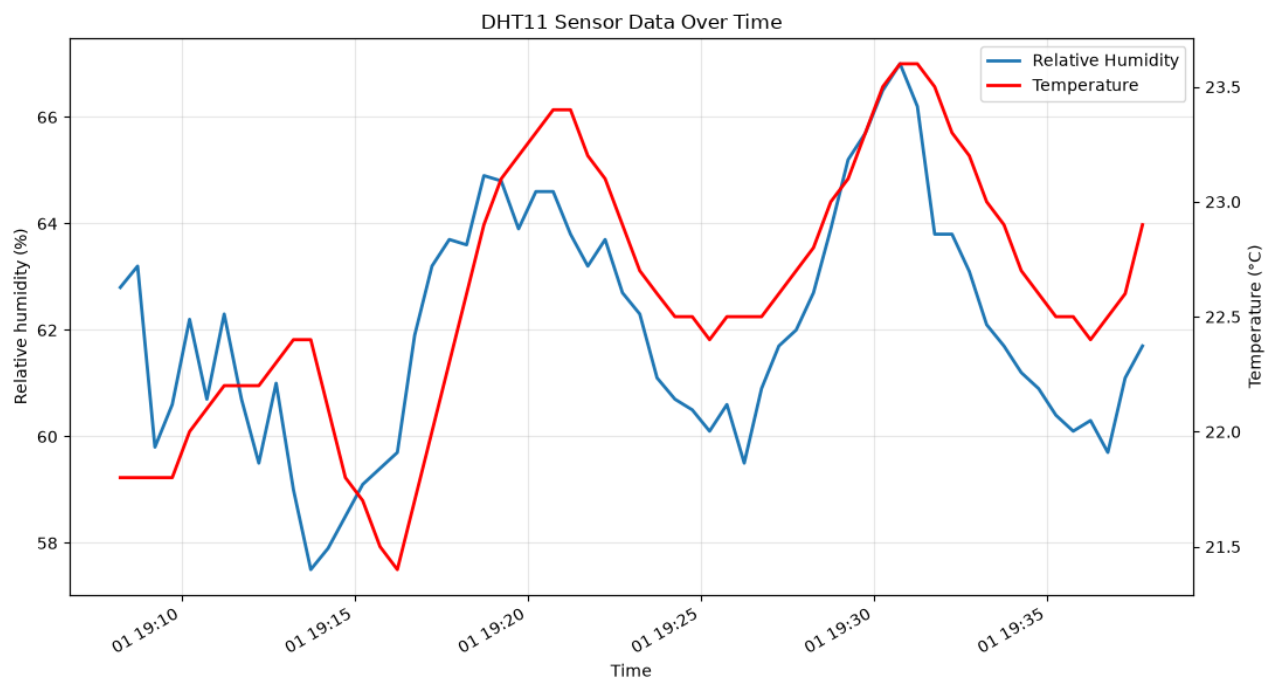
The data that is collected by the arduino from the sensors will be streamed over the serial port to the laptop. Where the python script will handle the temporary storage of all the data in memory. As previously mentioned before, a hash table will be used to collate all of the data together by sensor. The data will then be extracted from the hash table by key and transformed into the CSV format. This will then enable further analysis of the dataset and consequently visualisation of the data. However, some intermediate processing will occur before the data is written to the CSV file. Namely, outliers will be documented but removed from the dataset as this will inevitably skew the data. Further, any true/false values will be normalised to 1 or 0 values respectively. This will be done to ensure that the motion detection data can be displayed by count.

Data Quality Checking

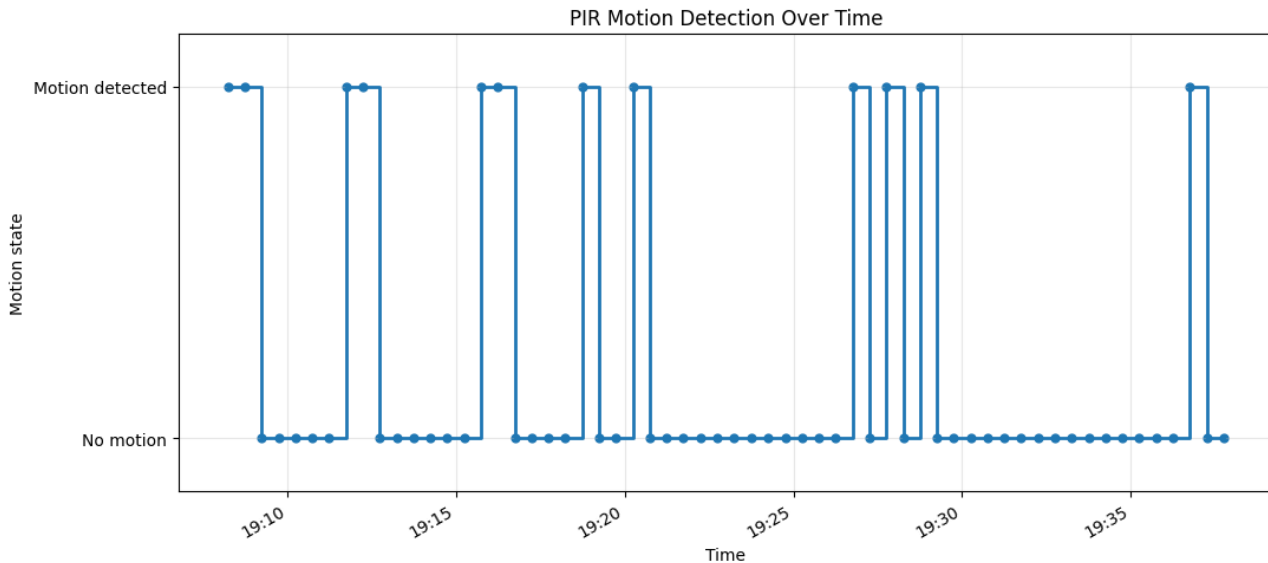
Upon collection and storage of the data the following data quality checks will be performed such as, handling missing or invalid data, these data points will be noted and then removed from the table. This is because keeping this data within the visualisation and the analysis of the visualisation will cause the data to become skewed. As a result of the data becoming skewed the analysis may become inaccurate or inferences that are being derived from the visualisation will be wrong. Moreover, outliers will also be removed from the visualisation themselves but will be noted within the analysis and reasoning will be provided as to why they may have occurred.

Analysis and Visualisation

Relative Humidity and temperature vs Time
Luminosity frequency



Passive Infrared Motion Detection



Patterns and key findings

Upon analysis of the DHT11 relative humidity and temperature data the following patterns can be seen, upon setup of the sensor there is a sharp decline in both humidity and temperature values. This is then followed by a rise in both temperature and relative humidity over a period of 5 minutes. This is then followed by a subsequent trough before another crest within the next 5 minute interval. This trend appears to continue in at the point where the data collection ceased with another predicted rise. During the data collection period the relative humidity was stable, average 61.98% with a standard deviation of 2.18. The temperature was also stable averaging 22.59 degrees celsius with a standard deviation of only 0.56. The mean and median were similar for both measurements. This suggests that the data collected is balanced with no major outliers found. The range of the relative humidity was from 57.5% up to 67%, therefore having a total range of 9.5. Moreover, the temperature of the room ranged from 21.4 degrees celsius to 23.6 degrees. Thus, giving a range of only 2.2 degrees celsius. Both, relative humidity and temperature both reached their maximum values at the 30 minute mark.

While analysing the distribution of luminosity from the data given by the photo-resistor. It can be seen that the data has been clustered into two distinct groups. These being a group where low light conditions were detected (values between 38 – 79) and those where high levels of light (1000 - 1023) were detected. Moreover, there were 36 readings where the luminosity was above 1000 and this made up 60% of all readings within the collection period. Whereas, 24 readings, or 40% were between 38 and 79.

Further, upon analysing the data from the passive infrared motion sensor, it can be seen that there were multiple motion detection events over the collection period. Which correlate to the both the increase in relative humidity and temperature and increase in luminosity. However, motion was only detected in 12 of the 60 readings during the collection period. This may be due to the sensor only being read every 30 seconds which may have caused missed readings. The sensor may also have had low readings as it was positioned at the entry point and only sensed when people were either leaving or entering the space.

Interpretation and refinement

Given the DHT11 sensor data it is feasible, that we can determine when the bathroom space is occupied and being used. Especially, if they are utilising the shower. This is because utilising the shower increases both the temperature and humidity within the room and this is proven with the data which has been recently collected. Moreover, it is possible to determine the duration the room is being used for as the room during use and then gradually decreases as the room cools back to the ambient temperature. Further, it is possible to detect when the bathroom light is switched on or off which may signal that the area is about to be utilised. Both of these results are as expected and will directly assist in determining if the bathroom is being utilised, length, and approximation of time utilising the shower. Moreover, this data used in conjunction with the passive infrared motion sensor is able to show when and if the bathroom is in use. Therefore, providing a solution to the original problem that could be implemented. One improvement that should be made if utilising the sensor array is to integrate it with other smart home technologies such as flow meters. This will enable the device to be able to accurately determine usage of the bathroom space and also assist home owners curb their water usage.

Limitations

It is not possible to utilise the photo-resistor in daylight conditions to ascertain if someone is going to use the bathroom space or any space with high levels of natural light. Additional sensors such as the DHT11 which was used in this project are able to operate effectively at the time of day which this sensor is no longer useful. Further, this simulated collection period may not be representative of the typical usage of the space and further collection over a longer period is likely more accurate. Also, other factors may be present that can affect both humidity and temperature sensing from the DHT11 such as cleaning, or natural air entering the room. Another limitation is we cannot guarantee that the data collected by the passive infrared sensor is a person if the household contains pets such as dogs and cats.

Expectations

The trial demonstrates that the sensor array can capture changes associated with bathroom activity. The PIR sensor detected movement, while the luminosity and DHT11 sensors provided supporting information about changes in lighting, temperature and humidity. These results partially support the original expectation that the system could identify when bathroom activity was occurring. However, occupancy events were not formally classified or measured during the short controlled trial. Therefore, the system could not yet reliably determine when the bathroom was free or calculate how long it was occupied. The current system should be regarded as a proof of concept rather than a completed bathroom-availability system. Longer data collection, improved event-classification rules and additional supporting sensors would be required to meet the original expectations reliably.

References

HandsOn Technology n.d., *HC-SR501 passive infrared (PIR) motion sensor*, data sheet, viewed 28 July 2026,

<https://www.handsontec.com/dataspecs/SR501%20Motion%20Sensor.pdf>.

OSEPP Electronics n.d., *DHT11 humidity & temperature sensor*, data sheet, Mouser Electronics, viewed 28 July 2026, <https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf>.

SparkFun Electronics n.d., *GL5528 CdS photoconductive cells*, data sheet, viewed 28 July 2026,

<https://cdn.sparkfun.com/datasheets/Sensors/LightImaging/SEN-09088.pdf>.

The below code is added for reference the files have been uploaded to Cloud Deakin

Data Capture Evidence

```
Current Luminosity:
2026-07-27 19:28:45.242002,Luminosity,1023
Current humidity and temperature readings:
2026-07-27 19:28:45.267328,DHT11,63.90,23.00
PIR Sensor:
2026-07-27 19:28:45.267808,PIR,Motion Detected
Current Luminosity:
2026-07-27 19:29:15.269556,Luminosity,1023
Current humidity and temperature readings:
2026-07-27 19:29:15.295405,DHT11,65.20,23.10
PIR Sensor:
2026-07-27 19:29:15.296174,PIR,No Motion Detected
Current Luminosity:
2026-07-27 19:29:45.296244,Luminosity,1023
Current humidity and temperature readings:
2026-07-27 19:29:45.321822,DHT11,65.70,23.30
PIR Sensor:
2026-07-27 19:29:45.322401,PIR,No Motion Detected
Current Luminosity:
2026-07-27 19:30:15.323577,Luminosity,1023
Current humidity and temperature readings:
2026-07-27 19:30:15.349194,DHT11,66.50,23.50
PIR Sensor:
2026-07-27 19:30:15.349862,PIR,No Motion Detected
Current Luminosity:
2026-07-27 19:30:45.350698,Luminosity,1023
Current humidity and temperature readings:
2026-07-27 19:30:45.376379,DHT11,67.00,23.60
PIR Sensor:
2026-07-27 19:30:45.376944,PIR,No Motion Detected
Current Luminosity:
2026-07-27 19:31:15.377909,Luminosity,54
Current humidity and temperature readings:
2026-07-27 19:31:15.403335,DHT11,66.20,23.60
PIR Sensor:
2026-07-27 19:31:15.404003,PIR,No Motion Detected
Current Luminosity:
2026-07-27 19:31:45.404174,Luminosity,52
Current humidity and temperature readings:
2026-07-27 19:31:45.430244,DHT11,63.80,23.50
PIR Sensor:
2026-07-27 19:31:45.430453,PIR,No Motion Detected
Current Luminosity:
2026-07-27 19:32:15.431725,Luminosity,51
Current humidity and temperature readings:
2026-07-27 19:32:15.457763,DHT11,63.80,23.30
PIR Sensor:
2026-07-27 19:32:15.458331,PIR,No Motion Detected
Current Luminosity:
2026-07-27 19:32:45.458930,Luminosity,51
Current humidity and temperature readings:
2026-07-27 19:32:45.484693,DHT11,63.10,23.20
PIR Sensor:
2026-07-27 19:32:45.485269,PIR,No Motion Detected
Current Luminosity:
2026-07-27 19:33:15.486363,Luminosity,53
Current humidity and temperature readings:
2026-07-27 19:33:15.511975,DHT11,62.10,23.00
PIR Sensor:
2026-07-27 19:33:15.512545,PIR,No Motion Detected
Current Luminosity:
```

tput 🔧 Diagnostics 🔍 Search 📁 Project 🖨️ Terminal

20:44 INSERT Soft Tabs: 4 UTF-8 Python

Arduino Code

/SIT225-1P Sketch by Jordan Elkin/

```
#include <DHT.h>
```

```
//Constants that are used #define PHOTOCELL_PIN A0 #define DHT11_PIN 12 #define  
DHT_TYPE DHT11 #define BAUD_RATE 9600 #define PIR_PIN 5 #define START_DELAY  
120000 //set to 2 minutes as per the PIR spec sheet #define SAMPLING_DELAY_RATE 30000
```

/ Specifies the photoresistor object, and supporting methods and variables that are required. A class was used so that the values would be encapsulated within the object and to make setup easier by calling the required functions at object creation time and in the one place / class*

```
PIR{ public: PIR(int pin_number, int sampling_rate){ this->set_pin_number(pin_number); this->  
>set_sampling_rate(sampling_rate); arduino::pinMode(this->get_pin_number(),INPUT); } char  
get_status(){return this->status;}
```

```
/* read method gets the value from the PIR sensor, determines the status and returns the  
value / char read(){ int pin_reading = digitalRead(this->get_pin_number()); this->  
>set_status(pin_reading); return this->get_status(); } bool set_sampling_rate(int sampling_rate){ if  
(sampling_rate > 0){ this->sampling_rate = sampling_rate; return true; } return false; } bool  
set_status(int status){ if(status == LOW){ this->status = "No Motion Detected"; }else{ this->status  
= "Motion Detected"; } } int get_pin_number(void) {return this->pin_number;} void  
set_pin_number(int pin_number){this->pin_number = pin_number;}
```

```
private: char* status; int pin_number; int sampling_rate; };
```

*/*Specifies the photoresistor object, and supporting methods and variables that are required. A class was used so that the values would be encapsulated within the object and to make setup easier by calling the required functions at object creation time and in the one place */ class*

```
Photoresistor{ public: Photoresistor(int pin_number, int sampling_rate){ this->  
>set_pin_number(pin_number); arduino::pinMode(this->get_pin_number(),INPUT); this->  
>set_sampling_rate(sampling_rate); } bool set_pin_number(int pin_number){this->pin_number =  
pin_number; return true;} int get_pin_number(void){return this->pin_number;}
```

```
int get_luminosity(void){return this->luminosity_value;}  
void set_luminosity(int luminosity_value){this->luminosity_value =  
luminosity_value;}
```

```
int get_sampling_rate(void){return this->sampling_rate;}  
bool set_sampling_rate(int sampling_rate){  
if (sampling_rate > 0){  
    this->sampling_rate = sampling_rate;  
    return true;  
}  
return false;
```

} /read method gets the value/s from the photoresistor, sets the new value for the luminosity and returns this value/ int read(){ int pin_reading = analogRead(this->get_pin_number()); this->
>set_luminosity(pin_reading); return this->get_luminosity(); }

```
private: int pin_number; int luminosity_value; int sampling_rate; };
```

```

/variables and objects that are required for the lifetime of the program/ DHT
dht(DHT11_PIN,DHT_TYPE); float relative_humidity; float temperature; int rates[] =
{10,20,30}; bool startup; Photoresistor* resistor; PIR* pir;

void setup() { resistor = new Photoresistor(PHOTOCELL_PIN,10); pir = new
PIR(PIR_PIN,10); Serial.begin(BAUD_RATE); dht.begin(); // initialises the dht object startup =
true; }

//main loop which conducts the reading of the sensors and sends the data over the serial port to be //
collected by the endpoint void loop() { if(startup){ delay(15);// delay for serial
connection Serial.println("Setup in progress please wait approx 2 mins"); startup =
false; delay(START_DELAY); } int luminosity = resistor-
>read(); Serial.println(String("Luminosity") + "," + String(luminosity)); relative_humidity =
dht.readHumidity(); temperature = dht.readTemperature(); Serial.println(String("DHT11") + "," +
String(relative_humidity) + "," + String(temperature)); char* status = pir->read();
Serial.println(String("PIR") + "," + String(status)); delay(SAMPLING_DELAY_RATE);
}

```

Python Data Collection Code

```

import serial from datetime import datetime import time

BAUD_RATE = 9600

s = serial.Serial('/dev/ttyACM0', BAUD_RATE, timeout=5)

data_file = open("sensor_data.csv","w")

sensor_data = dict() #main sensor data store

#initial time information for determining how long data has been collected for start_time =
time.time()

#main loop for collecting data from the arduino over serial. while True: current_time = time.time()

if(abs(start_time - current_time) >= 30 * 60):
    break

data = s.readline()
data = data.decode('utf-8').strip("\r\n")
time_now = datetime.now().strftime("%H:%M:%S")

if(data.__contains__("Luminosity")):
    print(f'Current Luminosity:')
    print(f'{datetime.now()}, {data}')
    if "Luminosity" in sensor_data.keys():
        sensor_data['Luminosity'] += [f'{time_now}, {data}']
    else:
        sensor_data['Luminosity'] = [f'{time_now}, {data}']

if(data.__contains__("DHT11")):

```

```

print(f'Current humidity and temperature readings:')
print(f'{datetime.now()}, {data}')
if "DHT11" in sensor_data.keys():
    sensor_data['DHT11'] += [f'{time_now}, {data}']
else:
    sensor_data['DHT11'] = [f'{time_now}, {data}']

if(data.__contains__("PIR")):
    print(f'PIR Sensor:')
    print(f'{datetime.now()}, {data}')
    if "PIR" in sensor_data.keys():
        sensor_data['PIR'] += [f'{time_now}, {data}']
    else:
        sensor_data['PIR'] = [f'{time_now}, {data}']

#Writes data to the csv file in the specified csv format
data_file.write(f'DateTime,Sensor,value_1,value_2\n') # Headings
for key,value in sensor_data.items(): for inner_value in value: data_file.write(f'{inner_value}
\n') data_file.close()

```

Python Data Visualisation Code

```

import pandas as pd import matplotlib.pyplot as plt #load in sensor data as a dataframe
RELATIVE_HUMIDITY = 'value_1' TEMPERATURE = 'value_2'
LUMINOSITY = 'value_1'
df = pd.read_csv("sensor_data_complete.csv")
dht11 = df[df["Sensor"].astype(str).str.strip() == "DHT11"].copy()
#convert columns into correct data types dht11["DateTime"] =
pd.to_datetime(dht11["DateTime"],format="%H:%M:%S",errors="coerce") luminosity =
df[df["Sensor"].astype(str).str.strip() == "Luminosity"].copy()
#prepare data drop any false values that cannot be coerced
dht11[RELATIVE_HUMIDITY] = pd.to_numeric( dht11["value_1"], errors="coerce" )
dht11[TEMPERATURE] = pd.to_numeric( dht11["value_2"], errors="coerce" )
luminosity[LUMINOSITY] = pd.to_numeric( luminosity[LUMINOSITY], errors="coerce" )
#luminosity[LUMINOSITY] = luminosity.dropna(subset=[LUMINOSITY])
dht11 =
(dht11.dropna(subset=["DateTime",RELATIVE_HUMIDITY,TEMPERATURE]).sort_values("Date
Time"))
#creating graphs
fig,humidity_axis = plt.subplots(figsize=(11,6)) fig2, histogram_axis = plt.subplots(figsize=(10, 6))

```

```

#using a second vertical axis as humidity and temperature are in different units temperature_axis =
humidity_axis.twinx()

humidity_line =
humidity_axis.plot( dht11["DateTime"], dht11[RELATIVE_HUMIDITY], linewidth=2, linestyle="-", label="Relative Humidity" )

temperature_line =
temperature_axis.plot( dht11["DateTime"], dht11[TEMPERATURE], linewidth=2, linestyle="-", label="Temperature", color="r" )

histogram_axis.hist( luminosity["value_1"], bins=15, edgecolor="black" )

#Add graph labels humidity_axis.set_title("DHT11 Sensor Data Over Time") humidity_axis.set_xlabel("Time") humidity_axis.set_ylabel("Relative humidity (%)") temperature_axis.set_ylabel("Temperature (°C)")

histogram_axis.set_title("Distribution of Luminosity Readings") histogram_axis.set_xlabel("Luminosity reading") histogram_axis.set_ylabel("Frequency") histogram_axis.grid(axis="y", alpha=0.3)

#Add grid lines humidity_axis.grid(True, alpha=0.3)

#Combine both lines into one legend

lines = humidity_line + temperature_line labels = [line.get_label() for line in lines] humidity_axis.legend(lines, labels, loc="best")

#Format the time labels f
ig.autofmt_xdate()

#Prevent labels from being cut off
fig.tight_layout() fig2.tight_layout()

#Display the graph
plt.show()

```

Student name: Jordan Elkin

Student ID: 226621873

SIT225: Data Capture Technologies

Activity 1.1: Arduino Blink

Welcome to Arduino!

Arduino is an electronic prototyping platform. Different types of sensors & actuators can be attached to Arduino boards to create our own sensing-thinking-acting systems.

Throughout this unit, we will use Arduino to create different sensing devices, and to retrieve the collected sensor data.

In this task, we will try out an introductory exercise, to learn the basic concepts of Arduino.

Hardware Required

- Arduino Board with in-built LED

- USB cable

Software Required

- Arduino programming environment

Pre-requisites: You must do the following before this task

Unit site weekly materials

Active learning sessions require reading material and/or videos available in the unit site, which you must read/view **BEFORE** you start the lab. If you are on-campus, this means that we expect you to have gone through these materials when you join active learning sessions.

Why should you read/watch pre-lab materials?

These materials will help you understand the background which the lab tasks require. Students come to university from diverse backgrounds. Some of you may be familiar with the background information, some of you may not. When you come to the lab prepared, you're already equipped

with confidence and will be able to participate in activities better. Ultimately, class time will be much more productive, dynamic, and fun for everyone.

Here are the pre-lab materials for our first task:

1. Watch TED Talk: <https://www.youtube.com/watch?v=UoBUXOOdLXY> (~15 minutes)
2. Read Arduino tutorial: <https://www.dummies.com/article/technology/computers/hardware/arduino/how-to-complete-your-first-arduino-sketch-164747>)
3. Read this task sheet from beginning to end.

Task Objective

“We have an Arduino board with an in-built LED light. We need the LED light to be turned on and off continuously, every one second.”

Activity Submission Details

Answer the questions below in this word document and other activities in this activity sheet to create a PDF and submit to OnTrack as described in this week's OnTrack task. PDFs of this activity sheet and OnTrack task need to be merged for submission in the OnTrack portal.

Q1: The TED talk given under the Pre-Lab materials, shows how Arduino is being used for interesting projects to capture data from the environment, process it, and use it to carry out useful actions.

Fill the given table below to answer the following:

What are **three** projects that use captured data as given in the TED talk? What data do they capture? What sensors do you think they could use to capture this data?

Project name	Data captured	Sensors to capture the data
Pet Feeder	Recognition of different cats, proximity of cat to feeder.	RFID reader that identifies which cat has approached the feeder, Proximity sensors sense when cat is still within the pet feeder itself.
NYU/ITP (PS3 Controller)	Head movement, button click	IR webcam captures the position of the reflector on a baseball cap which then moves the cursor on the laptop in the same direction as the tracked movement. A mechanical button captures button clicks and when combined with the onscreen overlay allows for the pressing of on screen buttons with the mechanical button.

Sign Language glove	Hand flexion (i.e. finger position), x,y,z axis measurements	Flex sensor to determine the position of the fingers and workout the sign that is being made by the user which is then translated into text or speech. The accelerometer is detecting the position of the hand and its placement
---------------------	--	--

Q2: Consider the given Task Objective. Think about how this simple system can be decomposed to 'Sense-Think-Act'?

- a) What is the 'sensing' requirement in this system, if any?

There is a sensing requirement for this system because the task requires the light to be turned on and off and this should be verified. Since the LED light is an output device that turns on and off when a pin is set high (i.e. the voltage on that pin is 3.3v or 5v) or the pin is set low respectively, and to know that the LED light is indeed on we could add an additional sensing pin so that we can determine if the light has been turned on successfully or not. Therefore verifying the requirement that the LED light must turn on and off continuously every second and also sensing the state of the LED light.

- b) What is the 'thinking' requirement in this system, if any?

There is a thinking requirement in that we could record the amount of times that the pin is set high or low and analyse the data. But for this specific task the primary thinking requirement is to determine the state of the LED light from the information provided by a sensor which in this case is a sensing pin which will have a voltage when the light is on and no voltage when its off.

- c) What is the 'acting' requirement in this system, if any?

The primary acting requirement for this system is for the LED light to be turned on when it is off and off when it is on continuously every second.

Q3: Please refer to the provided 'Arduino Blink Activity Sheet' and follow the steps.

- a) In Arduino-speak, what is a "sketch"?

A sketch is the source code file for a program which runs on an Arduino. A sketch is usually written in C/C++ and is the way to write software that will run on the Arduino and interact with sensors and other devices with the Arduino.

- b) `setup()` and `loop()` are key Arduino constructs. These are required in every Arduino sketch.

- i) Which of the above two, runs once at the very beginning of your program and never again (unless you reset or upload new code)?

The `setup()` function runs once at the beginning of the program and sets up and initialises objects that are needed at runtime such as the Serial object/

- ii) Which of the above two, is used to continuously run code over and over again?

The `loop()` function runs continuously and causes the code written within to be executed forever until a new program is loaded into memory or the device is shutdown. The `loop()` function is run immediately after the `setup()` function has been executed and ended.

- c) What does **`pinMode()`** do?

The `pinMode()` function allows the developer to set the specified pins as an `INPUT`, `OUTPUT`, or `INPUT_PULLUP`. These specific modes allow for the pin to be set to high or low for output meaning that a voltage is now at the pin which allows for the actuating of led lights or a signal to be sent to a sensor connected at the pin. Whereas, the input mode allows for the reading of a pin which means monitoring for changes such as High → low or low → high. This also enables sensors to send data to the pin by setting the input pin as either low or high (0 or 1) and sending the data bit by bit.

Hint: <http://arduino.cc/en/Reference/HomePage>

- d) What is a comment?

A comment which can be multi-line or single line `/**/` or `//` respectively, allows the developer to add additional explanation for code function when it is not clear why the code is written a certain way or its not clear as to why certain constants are set the way that they are.

- e) What does the following line of code do:

```
delay(x);
```

This method causes the Arduino to sleep or wait for a specific amount of seconds which in the above example is the x argument.

Hint: <http://arduino.cc/en/Reference/HomePage>

- f) There is something you need to check before uploading your sketch. What is this?

Q4: How can you test the Blink program to make sure it is working as given in the Task Objective?

Firstly, you can run the verify option in the IDE. This will check for any errors that may cause the program to not function as required in the task objective. Secondly, you could alter the blink program and attach a new LED light to a pin and then set the pin high and low every second which would verify that the Blink program is functioning as intended. Thirdly, you can check where the inbuilt LED is on the Arduino and time it and check that it is indeed turning off and on continuously every second. Lastly, you could have the Arduino write to the serial port every second that it turns the light on and off and with these debug messages you could cross reference with the physical device and check that the light is matching up with the serial messages.

Activity 1.2: Write Arduino data to serial communication port

Now you can blink Arduino's built-in LED, it is time to talk to outside Arduino-world, your computer which connects the Arduino board using a USB cable. Arduino IDE shows what you write to the serial port.

Hardware Required

Arduino Board with in-built LED

USB cable

Software Required

Arduino programming environment

Steps:

Steps	Actions
1	Identify the port through which Arduino is connected to your computer. You can find it in Arduino IDE Tools menu. Write Arduino sketch (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/sketch_blink.ino), which look like below, in Arduino IDE, deploy in your board and observe the output in IDE's serial monitor.

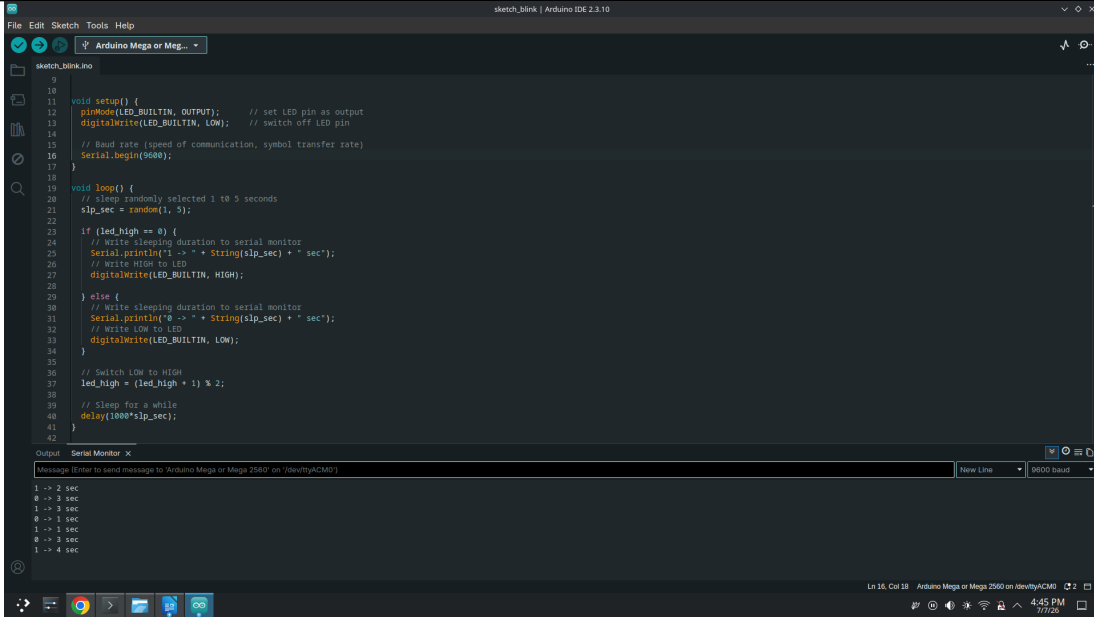
```

1  int led_high = 0; // variable to flag ON (1) or OFF (0)
2  int slp_sec = 0; // variable holds number of seconds to sleep
3
4
5  void setup() {
6      pinMode(LED_BUILTIN, OUTPUT); // set LED pin as output
7      digitalWrite(LED_BUILTIN, LOW); // switch off LED pin
8
9      // Baud rate (speed of communication, symbol transfer rate)
10     Serial.begin(6900);
11 }
12
13 void loop() {
14     // sleep randomly selected 1 to 5 seconds
15     slp_sec = random(1, 5);
16
17     if (led_high == 0) {
18         // Write sleeping duration to serial monitor
19         Serial.println("1 -> " + String(slp_sec) + " sec");
20         // Write HIGH to LED
21         digitalWrite(LED_BUILTIN, HIGH);
22     } else {
23         // Write sleeping duration to serial monitor
24         Serial.println("0 -> " + String(slp_sec) + " sec");
25         // Write LOW to LED
26         digitalWrite(LED_BUILTIN, LOW);
27     }
28
29     // Switch LOW to HIGH
30     led_high = (led_high + 1) % 2;
31
32     // Sleep for a while
33     delay(1000*slp_sec);
34 }
35
36

```

Question: Screenshot the serial monitor output and paste the image here.

Answer: Please see image below

	
2	Question: Observe the use of “Serial” such as functions Serial.begin() in setup and Serial.println() in loop. Describe what these functions are doing with respect to the serial monitor output you have attached above. Answer: The Serial being method initialises the serial monitor connection and establishes the baud rate which in the screenshot above I set to 9600. The Serial.println() method in the loop above writes to the serial monitor a string which has been passed to the method and is then displayed on the serial monitor console in the Arduino IDE.
3	Question: If Arduino transfers data at 4800 bits per second (baud rate) and you're sending 12 bytes of data, how long does it take to send over this information? Answer: If the baud rate is 4800 bits per second and 12 bytes of data is sent then it will take approximately 0.02 seconds (20 ms) for the data to be sent. This is calculated as shown below. The calculation is only an approximation as it does not account for any padding bits or parity bits. baud rate = 4800 data = 12 bytes => 96 bits transfer speed = 4800 bits / second => 4.8 bits / ms 96 / 4.8 = 20 ms

Activity 1.3: Arduino talks to Python

To listen to what Arduino sends, there will be a Python program running and keep listening to the same port where Arduino is writing data to receive it.

Hardware Required

Arduino Board with in-built LED

USB cable

Software Required

Arduino programming environment

Python 3.0 (Follow Python installation manual in unit site)

Steps:

Steps	Actions
1	Write Arduino sketch (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/sketch_serial_comm.ino) which looks like below. Open in Arduino IDE. Study the code. Upload the code to Arduino board and observe output in Arduino IDE serial monitor.

```

1
2 int x;
3
4 void setup() {
5     Serial.begin(9600); // set baud rate
6 }
7
8 void loop() {
9     while (!Serial.available()) {} // wait for data to arrive
10
11     // read string data from Serial, we know Python
12     // script is sending just an integer.
13     x = Serial.readString().toInt();
14
15     // write a string (no newline)
16     Serial.print("Arduino sends: ");
17
18     // Add 1 to what received.
19     // Write an integer with a newline (println vs print).
20     Serial.println(x + 1);
21
22     // Push the data through serial channel.
23     Serial.flush();
24 }

```

Question: Do you see any output in serial monitor? If not, then why?

Answer: Yes you see output in the serial monitor when you send information over the serial connection to the arduino. The arduino is currently only set to expect an integer value which it will then add 1 to and then write to the serial monitor through the serial objects print method. However, merely opening the serial monitor would not cause any output to appear.

- 2 Write Python code as below (or download from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_1/serial_comm_script.py) and save it to a file serial_comm_script.py.

```

1 import serial
2 import random
3
4 # set baud rate, same speed as set in your Arduino sketch.
5 boud_rate = 9600
6
7 # set serial port as suits your operating system
8 s = serial.Serial('/dev/ttyACM1', boud_rate, timeout=5)
9
10 while True: # infinite loop, keep running
11
12     # a random number between 5 and 50.
13     data_send = random.randint(5, 50)
14
15     # write to serial port, set data encoding.
16     # Raw bytes are sent through serial ports, Python bytes() needs
17     # to know the encoding to generate bytes from string.
18     #
19     # We send a single integer which is read from Arduino sketch.
20     #
21     d = s.write(bytes(str(data_send), 'utf-8'))
22     print(f"Send >>> {data_send} ({d} bytes)")
23
24     # Read from serial port.
25     #
26     # readline keeps reading until a newline found in the data stream.
27     # Unlike write above, we just send an integer with no newline.
28     # You should receive data the same way as it is sent.
29     #
30     d = s.readline().decode("utf-8")
31     print(f"Recv <<< {d}")
32

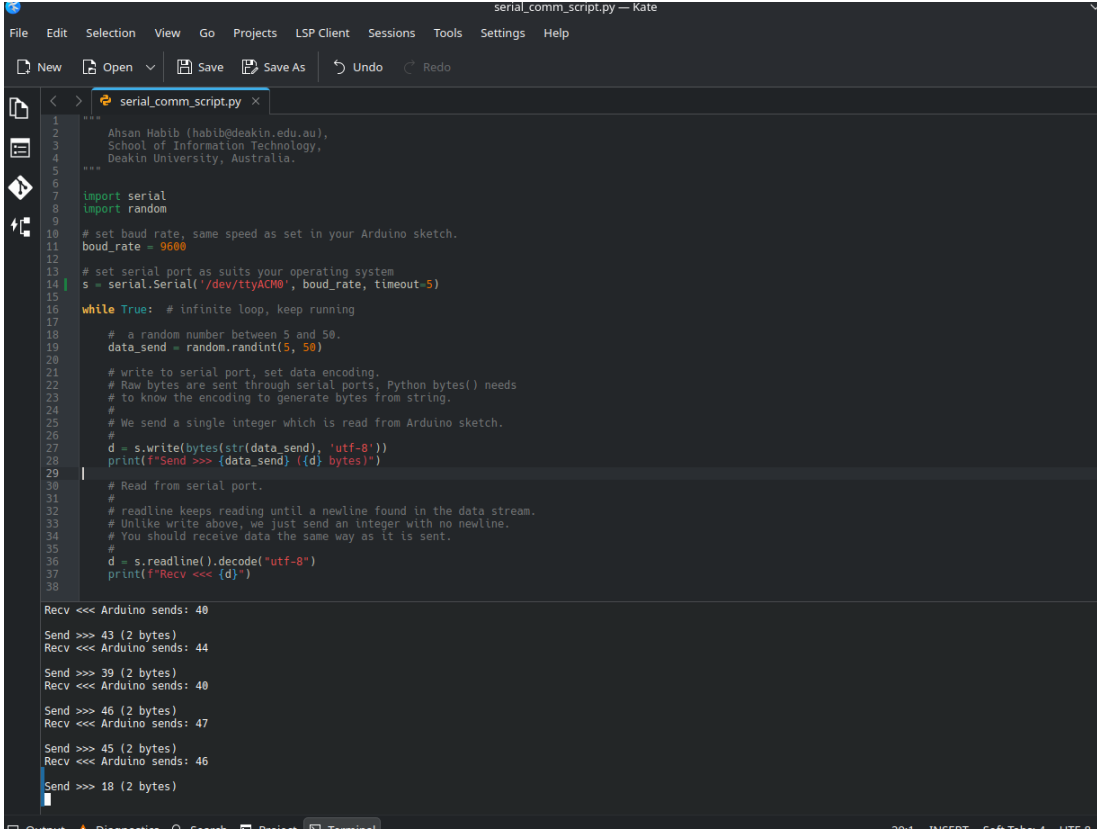
```

Run the Python file from command line using command:

\$ *python serial_comm_script.py*

Question: Run Python script in command line, does the script run or do you receive any error? If there is an error, analyse the error message and identify what went wrong.

Answer: initially the script did not run because the serial port on my computer was a different number than the one hard-coded in the script. Further, since I had the arduous IDE running with its serial monitor opened at the same time the resource was busy so no serial communications could start. However, after making the necessary changes to the script and closing the Arduino IDE I was able to connect to the Arduino over the serial interface and the program was able to send and receive the data no problem.

3	Question: Following the above step 2, do you think Arduino IDE is keeping the serial communication port busy talking to Arduino board? Now try to close the Arduino IDE and repeat step 2 above to run the Python script. Do previous errors show up again? If not, Python script should have print messages in the command line. Take the screenshot of the Python script output and paste here. Answer: Yes the Arduino IDE is keeping the port busy when its serial monitor is opened. As previously stated these issues were corrected once the serial monitor is closed or the IDE is closed down completely, either option results in the serial interface being opened.  <pre> 1 """ 2 Ahsan Habib (ahabib@deakin.edu.au), 3 School of Information Technology, 4 Deakin University, Australia. 5 """ 6 7 import serial 8 import random 9 10 # set baud rate, same speed as set in your Arduino sketch. 11 baud_rate = 9600 12 13 # set serial port as suits your operating system 14 s = serial.Serial('/dev/ttyACM0', baud_rate, timeout=5) 15 16 while True: # infinite loop, keep running 17 18 # a random number between 5 and 50. 19 data_send = random.randint(5, 50) 20 21 # write to serial port, set data encoding. 22 # Raw bytes are sent through serial ports; Python bytes() needs 23 # to know the encoding to generate bytes from string. 24 # 25 # We send a single integer which is read from Arduino sketch. 26 # 27 d = s.write(bytes(str(data_send), 'utf-8')) 28 print(f"Send >>> {data_send} ({d} bytes)") 29 30 # Read from serial port. 31 # 32 # readline keeps reading until a newline found in the data stream. 33 # Unlike write above, we just send an integer with no newline. 34 # You should receive data the same way as it is sent. 35 # 36 d = s.readline().decode("utf-8") 37 print(f"Recv <<< {d}") 38 39 Recv <<< Arduino sends: 40 40 Send >>> 43 (2 bytes) 41 Recv <<< Arduino sends: 44 42 Send >>> 39 (2 bytes) 43 Recv <<< Arduino sends: 40 44 Send >>> 46 (2 bytes) 45 Recv <<< Arduino sends: 47 46 Send >>> 45 (2 bytes) 47 Recv <<< Arduino sends: 46 48 Send >>> 18 (2 bytes) </pre>
4	Question: Observe the Python script output and describe the communication protocol used between Arduino sketch and Python script. Answer: The communication protocol that is occurring is known as the serial communication. There are multiple types of serial communications these include SPI, I2C and UART. By default the serial interface for the Arduino is the Universal asynchronous receiver-transmitter (UART). Where one pin is used for transmission and the other is for receiving on. In the script the Arduino is sent a string in the form of raw bytes which is a utf-8 encoding. This means that characters are encoded in a

	series of 8 bits for ASCII characters and up to 4 bytes for other languages. These bits are then transmitted to the Arduino one bit at a time and then the inverse is conducted when the Arduino writes to the serial interface and the python program reads the bytes it receives from the Arduino stopping only when a newline character is encountered which triggers the contents being printed to the console.
--	--

Student name: Jordan Elkin

Student ID:226621873

SIT225: Data Capture Technologies

Activity 2.1: Working with sensor - DHT22

DHT22 is a temperature and humidity sensor.

Hardware Required

Arduino Board

DHT22 sensor

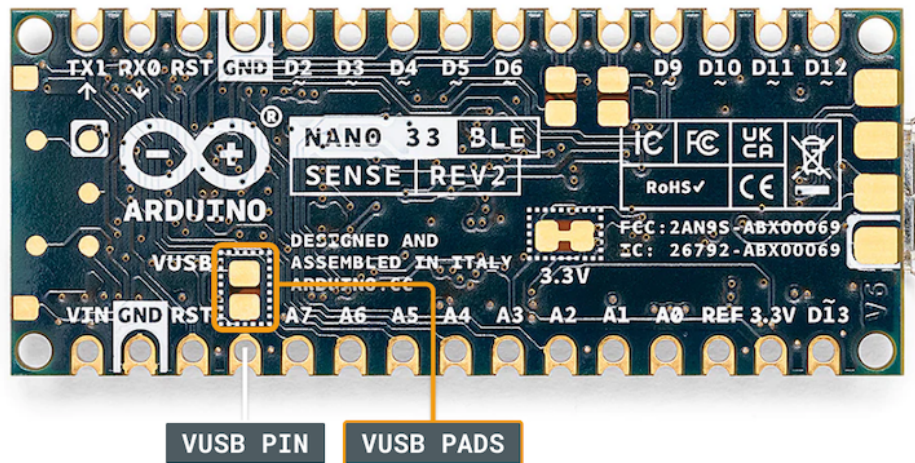
USB cable

Software Required

Arduino programming environment

Known issue, action required

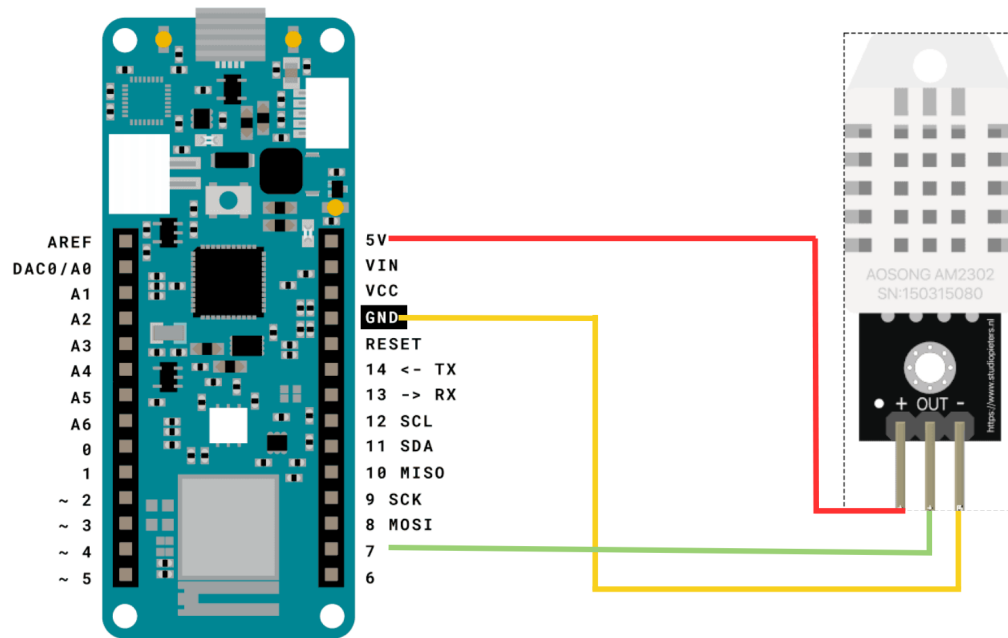
Arduino Nano 33 IoT board operates on 3.3 V, it needs to be arranged to make it 5 V. The Arduino board has a pin called VUSB or VBUS and there are 2 pads next to the pin. **These two pads must be shorted to enable the pin** (see detail here <https://support.arduino.cc/hc/en-us/articles/360014779679-Enable-5-V-power-on-the-VUSB-or-VBUS-pin-on-Nano-boards>).



To test, you can use a wire and connect these 2 pads manually by hand to see if data is coming through DHT22 sensor. For long data collection, you will need to solder the wire permanently to connect the pads. Seeking help from tutors there is an on-campus facility called Maker Space where you can do it.

Steps:

Step	Action
1	Connect your DHT22 Temperature and Humidity Sensor to the Arduino board. Note that the pin layout in the image below may look different than the board you may have.



- Pick a red male-female jumper wire and attach the female end to pin 1 (VCC pin) on the sensor. Plug the male end into the Arduino board's 5V power pin.
- Pick a blue male-female jumper wire and attach the female end to pin 2 (DATA pin) on the sensor. Plug the male end into the Arduino board's digital data pin 2.
- Pick a black male-female jumper wire and attach the female end to pin 4 (GND) on the sensor. Plug the male end into the Arduino board's GND pin.
- Sensor's pin 3 is not used.

2 Connect your Arduino board to your computer using the USB cable.

3 Write an Arduino sketch (or download it from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_dht22.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

```

6
7 // Add "DHT sensor library"
8 #include <DHT.h>
9
10 #define DHTPIN 2 // digital pin number
11 #define DHTTYPE DHT22 // DHT type 11 or 22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // variables to store data.
15 float hum, temp;
16
17 void setup() {
18     // Set baud rate for serial communication
19     Serial.begin(9600);
20
21     // initialise DHT library
22     dht.begin();
23 }
24
25 void loop() {
26     // read data
27     hum = dht.readHumidity();
28     temp = dht.readTemperature();
29
30     // Print data to serial port - a long way
31     //
32     // Serial.print("Humid: ");
33     // Serial.print(hum);
34     // Serial.print(" %, Temp: ");
35     // Serial.print(temp);
36     // Serial.println(" Celsius");
37
38     // Print data to serial port - a compact way
39     Serial.println(String(hum) + "," + String(temp));
40
41     // wait a while
42     delay(15*1000);
43 }

```

4 **Question:** A spec of the DHT22 sensor is given in the link below. It mentions that the sampling rate is 0.5 Hz.

<https://lastminuteengineers.com/dht11-dht22-arduino-tutorial>

- i) What does the sampling rate mean?
- ii) Where is this used in the Arduino code?

Answer:

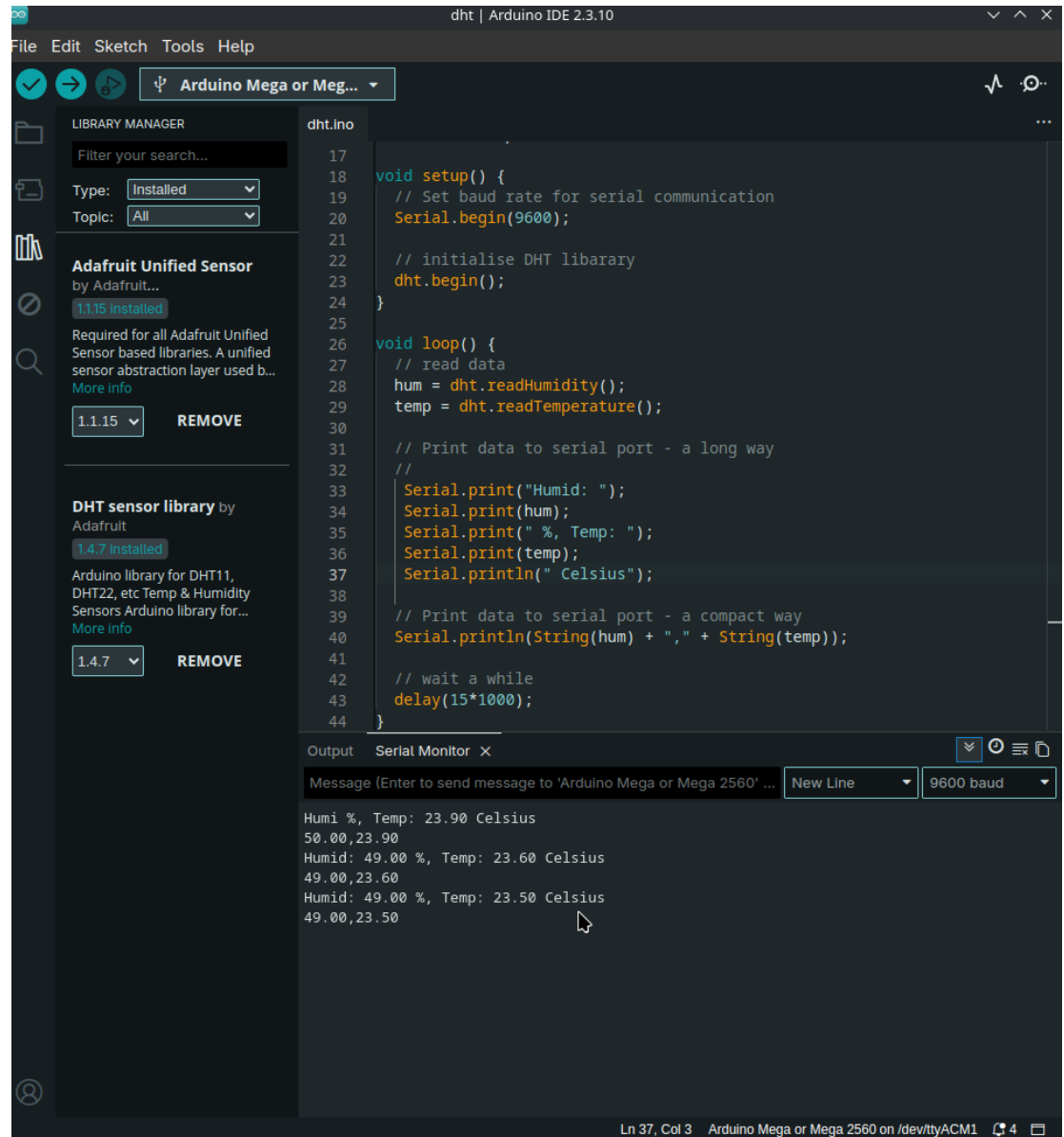
i) The sampling rate means the amount of times per second that the Arduino will receive input data from the sensor. This means that the sensor will only output new temperature and humidity readings every 0.5 seconds and if the sensor is polled before this time it will not have data available..

ii) Sampling in the above code is done in the method readHumidity() and readTemperature(). The standoff time is the delay which means

that the loop repeats when the sensor is ready to output new data.

- 5 **Question:** Take a screenshot of your Serial Monitor displaying temperature & humidity sensor data logs. Add the image here.

Answer:



Activity 2.2: Working with sensor - HC-SR04

HC-SR04 is an Ultrasonic sensor.

Hardware Required

Arduino Board

HC-SR04 Ultrasonic sensor

USB cable

Software Required

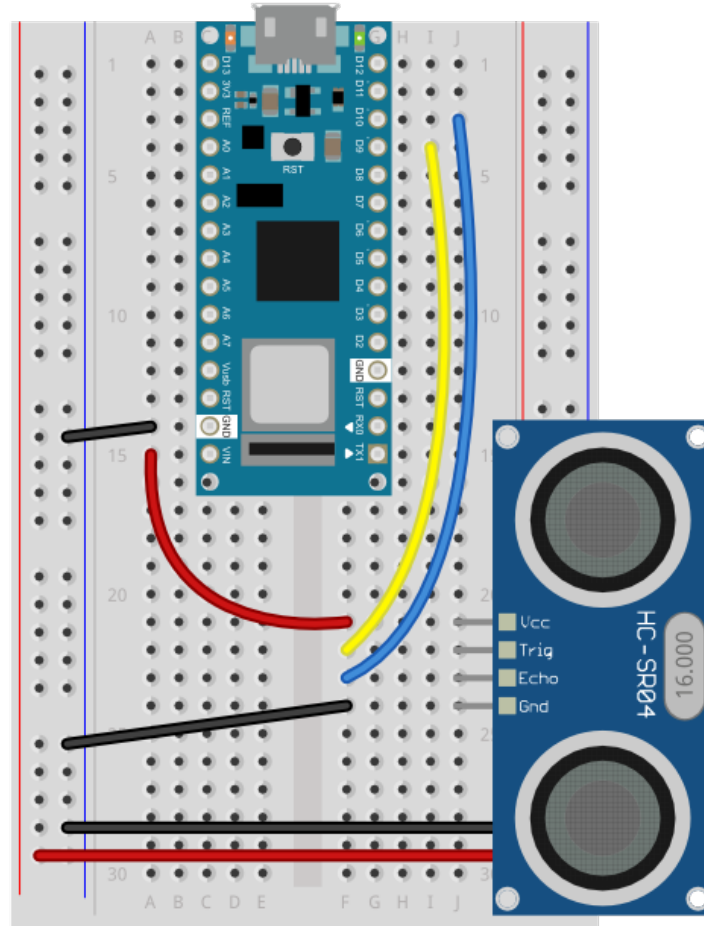
Arduino programming environment

Known issue, action required

The same known issue applies to SR04 which operates at 5 V source while the Arduino Nano 33 IoT supplies 3.3 V. It requires 2 VUSB (or VBUS) pads to be shorted to enable the pin.

Steps:

Step	Action
1	Connect your HC-SR04 sensor to the Arduino board. Note that the pin layout in the image below may look different than the board you may have.



2 Connect your Arduino board to your computer using the USB cable.

3 Write an Arduino sketch (or download it from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_hcsr04_distance.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

```

7  const int trigger = 2;
8  const int echo = 3;
9
10 int getUltrasonicDistance(){
11     // Function to retrieve the distance reading of the ultrasonic
12     long duration;
13     int distance;
14
15     // Assure the trigger pin is LOW:
16     digitalWrite(trigger, LOW);
17     // Brief pause:
18     delayMicroseconds(5);
19
20     // Trigger the sensor by setting the trigger to HIGH:
21     digitalWrite(trigger, HIGH);
22     // Wait a moment before turning off the trigger:
23     delayMicroseconds(10);
24     // Turn off the trigger:
25     digitalWrite(trigger, LOW);
26
27     // Read the echo pin:
28     duration = pulseIn(echo, HIGH);
29     // Calculate the distance in centimeter (CM):
30     distance = duration * 0.034 / 2;
31
32     // Uncomment this line to return value in IN instead of CM:
33     //distance = distance * 0.3937008
34
35     // Return the distance read from the sensor:
36     return distance;
37 }
38
39 void setup() {
40     // Define inputs and outputs:
41     pinMode(trigger, OUTPUT);
42     pinMode(echo, INPUT);
43
44     // Start the serial monitor:
45     Serial.begin(9600);
46 }
47
48 void loop() {
49     // Print the distance to the serial monitor:
50     Serial.print("Distance: ");
51     Serial.println(getUltrasonicDistance());
52
53     // Wait one second before continuing:
54     delay(1000);
55 }

```

4

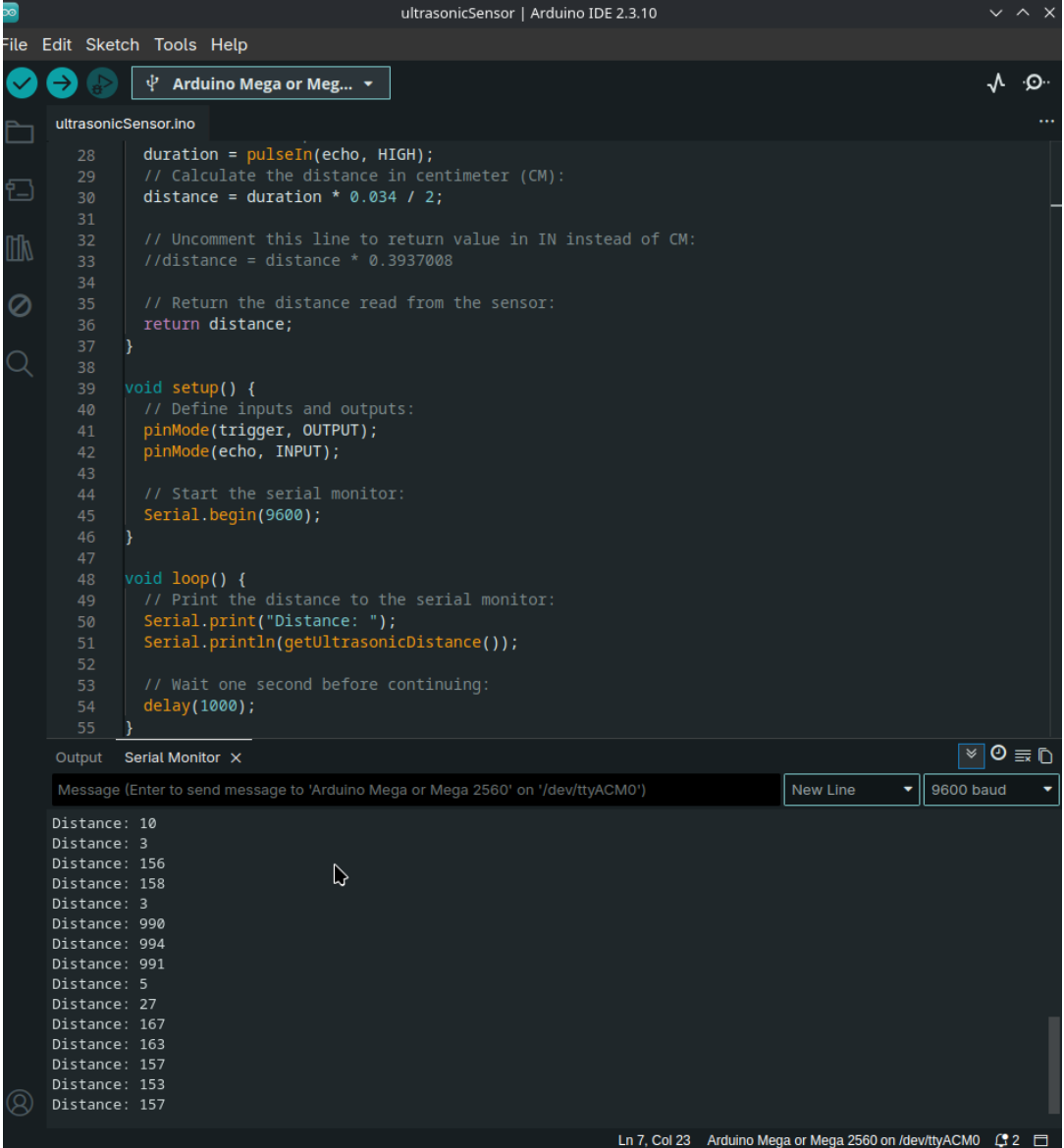
Question: Spec of SR04 is available here (<https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf>). Identify 2 critical aspects you should be careful about this sensor operation.

Answer:

1)The sensor operates at 5v DC and has a maximum working current of 15mA. Exceeding these will damage the device.

2) The module automatically send eight 40khz pulses to detect whether there is a signal back. If the signal does come back the distance needs to be halved to measure the distance an object is from the sensor not there and back. Further, the trigger input pulse is 10 microsecond pulse.

- 5 **Question:** Take a screenshot of your Serial Monitor displaying distance values while you try to generate a periodic motion by moving your hand gradually back and forth towards the sensor. Add the image here.



The screenshot displays the Arduino IDE interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. Below the menu is a toolbar with icons for checking, running, and uploading code, along with a dropdown menu for the board type, currently set to 'Arduino Mega or Meg...'. The main editor window shows the code for 'ultrasonicSensor.ino'. The code defines a function to calculate distance from an ultrasonic sensor, with comments in Italian. It includes a setup function to initialize pins and the serial monitor, and a loop function to continuously read and print the distance. The Serial Monitor window at the bottom shows a list of distance readings in centimeters, such as 10, 3, 156, 158, 3, 990, 994, 991, 5, 27, 167, 163, 157, 153, and 157. The status bar at the bottom indicates 'Ln 7, Col 23' and 'Arduino Mega or Mega 2560 on /dev/ttyACM0'.

```
ultrasonicSensor.ino
28 duration = pulseIn(echo, HIGH);
29 // Calculate the distance in centimeter (CM):
30 distance = duration * 0.034 / 2;
31
32 // Uncomment this line to return value in IN instead of CM:
33 //distance = distance * 0.3937008
34
35 // Return the distance read from the sensor:
36 return distance;
37 }
38
39 void setup() {
40 // Define inputs and outputs:
41 pinMode(trigger, OUTPUT);
42 pinMode(echo, INPUT);
43
44 // Start the serial monitor:
45 Serial.begin(9600);
46 }
47
48 void loop() {
49 // Print the distance to the serial monitor:
50 Serial.print("Distance: ");
51 Serial.println(getUltrasonicDistance());
52
53 // Wait one second before continuing:
54 delay(1000);
55 }
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino Mega or Mega 2560' on '/dev/ttyACM0')

New Line 9600 baud

Distance: 10
Distance: 3
Distance: 156
Distance: 158
Distance: 3
Distance: 990
Distance: 994
Distance: 991
Distance: 5
Distance: 27
Distance: 167
Distance: 163
Distance: 157
Distance: 153
Distance: 157

Ln 7, Col 23 Arduino Mega or Mega 2560 on /dev/ttyACM0 2

Answer:

--	--

Activity 2.3: Working with sensor - Accelerometer

LSM6DS3 module on the Arduino Nano 33 IoT is an accelerometer and gyroscope sensor.

Hardware Required

Arduino Nano 33 IoT Board (has inbuilt LSM6DS3 module)

USB cable

Software Required

Arduino programming environment

Steps:

Step	Action
1	Write Arduino sketch (or download code from https://github.com/deakin-deep-dreamer/sit225/blob/main/week_2/sketch_accelero.ino) which looks like below. Compile the code in Arduino IDE, deploy to the board and observe output in the Arduino IDE serial monitor.

```

6
7 // Add Arduino_LSM6DS3 library
8 // from Arduino IDE Library Manager.
9 #include <Arduino_LSM6DS3.h>
10
11 float x, y, z;
12
13 void setup() {
14     Serial.begin(9600); // set baud rate
15     while (!Serial); // wait for port to init
16     Serial.println("Started");
17
18     if (!IMU.begin()) {
19         Serial.println("Failed to initialize IMU!");
20         while (1);
21     }
22
23     Serial.println(
24         "Accelerometer sample rate = "
25         + String(IMU.accelerationSampleRate()) + " Hz");
26 }
27
28 void loop() {
29     // read accelero data
30     if (IMU.accelerationAvailable()) {
31         IMU.readAcceleration(x, y, z);
32     }
33
34     Serial.println(
35         String(x) + ", " + String(y) + ", " + String(z));
36
37     delay(1000);
38 }

```

2

Question: Spec of LSM6DS3 is available here (https://content.arduino.cc/assets/st_imu_lsm6ds3_datasheet.pdf).

Identify at least 3 attributes of this sensor you think important to work with.

Answer:

1) The accelerometer as this can give detailed information about if the device is falling or changing directions. The accelerometer has many real world applications where it would be beneficial such as identifying if a patient has fallen as a sharp change in acceleration would then make that known. Further, an accelerometer could be used to identify if people have crashed their cars when driving this is because the acceleration would be constant and very quickly stop.

2) The gyroscope used in conjunction with the accelerometer will ensure accurate calculations of direction of travel. This is because gyroscopes calculate angular

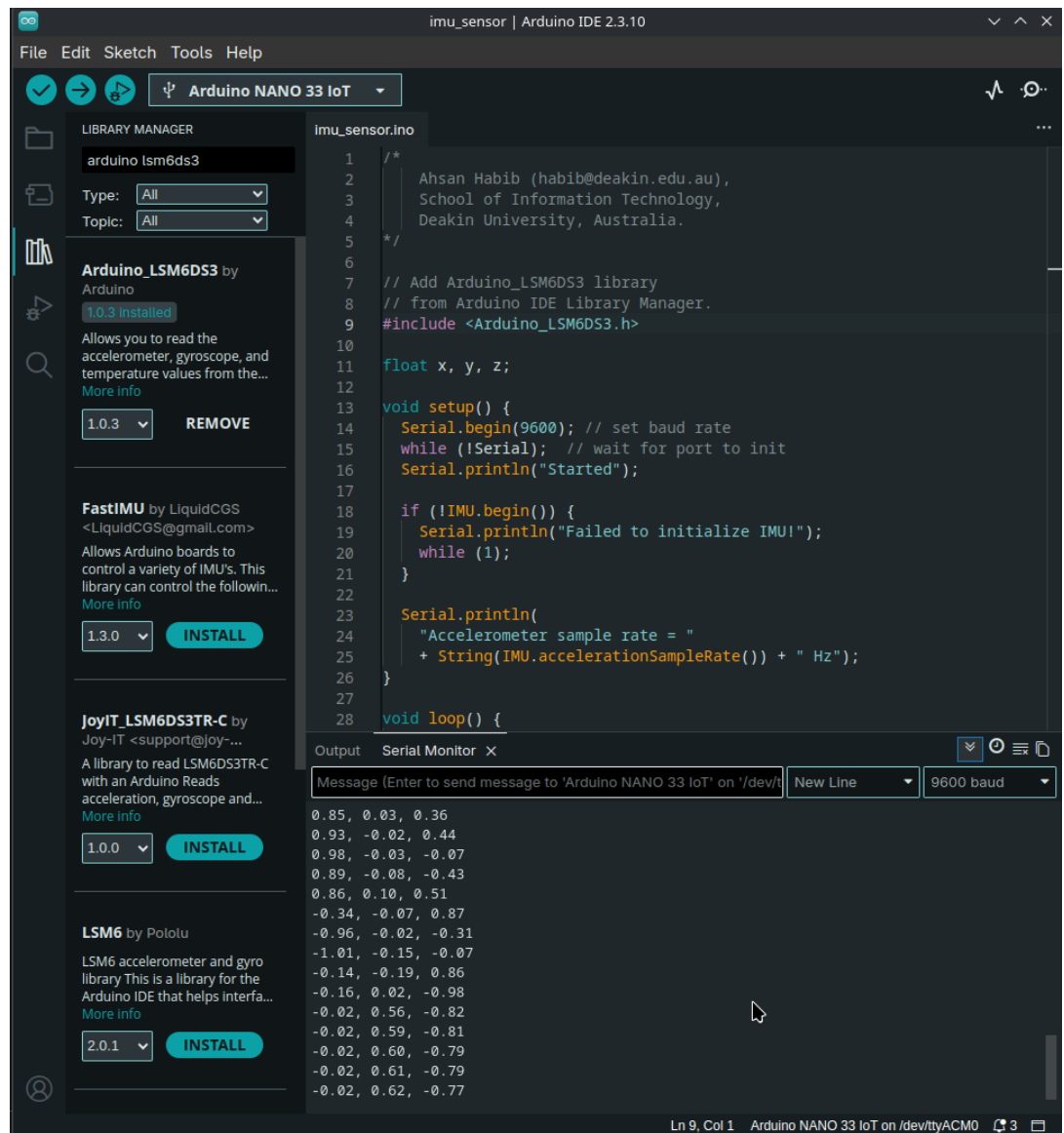
momentum and when combined with the information provided by the accelerometer, direction can be ascertained.

3) The free-fall interrupt which is a builtin feature is very useful as this will allow the device to be used in many contexts where finding out if someone or something is falling. Such as a person falling from height when working at heights, patients falling, or a controller that is controlling heavy machinery has fallen out of the operators hands and the machine must immediately stop functioning.

3

Question: Take a screenshot of your Serial Monitor displaying sensor readings. Add the image here.

Answer:



4	Question: Identify the max sampling rate and consider reducing the delay (line 37 in the sketch) to increase the number of samples. Summarise your findings here. Answer: The max sampling rate of the IMU is as quick as the chip can go i.e. the clock speed of the device. As the delay decreased to 0 the results generated by the device changed almost instantly as the device was rotated in 3D space. Upon checking the documentation for the arduino nano 33 iot, it stated that the device is configured by default with the library at 104hz which means that we can receive a new sample every 9.6 milliseconds. However, the true maximum sampling rate of the LSM6DS3 for the accelerometer is 6 664 Hz and 1 666 Hz for the gyroscope. This means if set to a higher rate we could receive an enormous amount of samples in less than a second. That is at least $1 / 1\,666$ samples a second or a sample every 0.6 milliseconds.

Activity 2.4: Plot data using Python Notebook

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. You can find detail in official website (<https://matplotlib.org>).

Hardware Required

No hardware required

Software Required

Python Jupyter notebook

Steps:

Step	Action
1	Download the Jupyter notebook week2_notebook.ipynb from here (https://github.com/deakin-deep-dreamer/sit225/blob/main/week 2/week2_notebook.ipynb). Follow the instructions in the notebook to carry out instructions and finally convert the notebook to PDF so you can combine it with this activity sheet PDF.

week2_notebook-Matplot lib- Jordan Elkin

July 28, 2026

```
[ ]: student_name = "Jordan Elkin " # fill your name
      student_id = "226621873" # fill your student ID
      print("Student name: " + student_name)
      print("Student ID: " + student_id)
```

Student name:

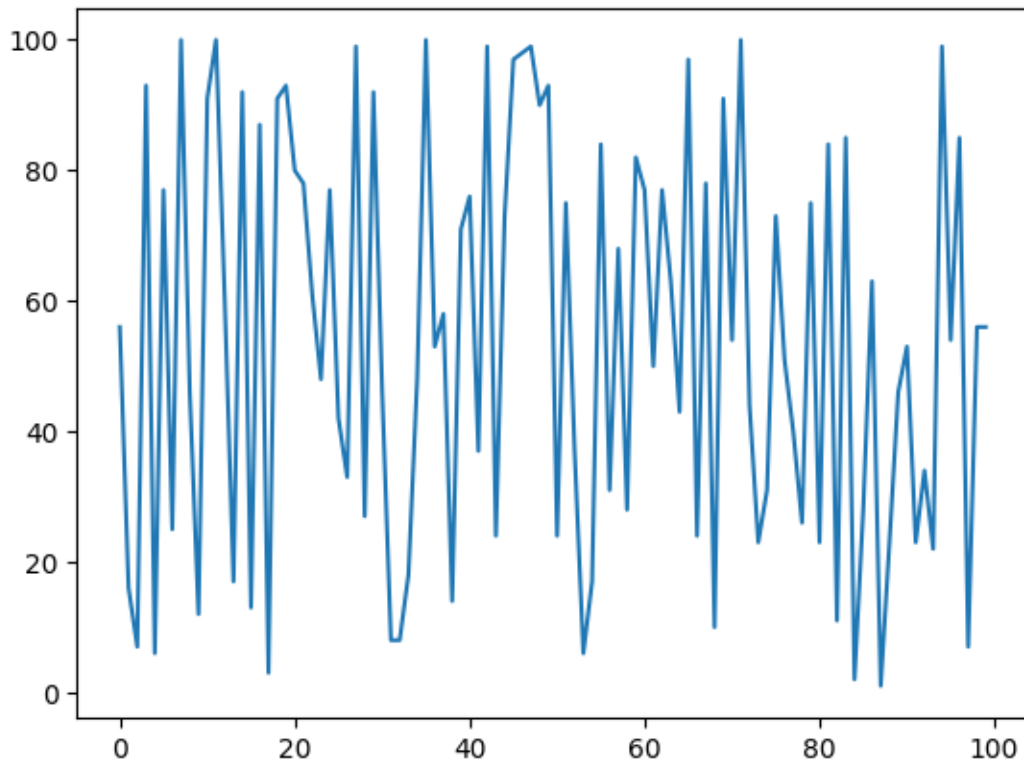
Student ID:

```
[3]: import random
      import matplotlib.pyplot as plt

      n_values = 100
      y_values = []

      # Create data (y_values) randomly between 1 and 100.
      for i in range(n_values):
          y_values.append(random.randint(1, 100))

      x_values = range(n_values) # X is sequence of values 0-99
      plt.plot(x_values, y_values)
      plt.show()
```

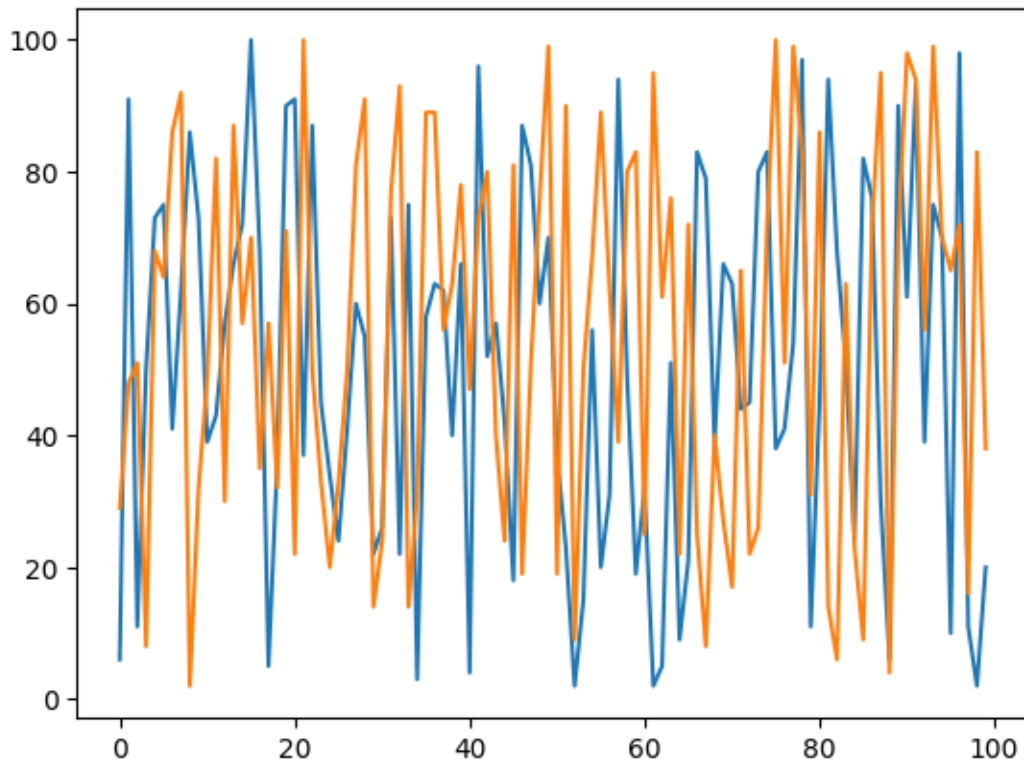


```
[ ]: # Plot 2 variables
#

n_values = 100
y_values_1 = []
y_values_2 = []

# Create data (y_values) randomly between 1 and 100.
for i in range(n_values):
    y_values_1.append(random.randint(1, 100))
    y_values_2.append(random.randint(1, 100))

x_values = range(n_values) # X is sequence of values 0-99
plt.plot(x_values, y_values_1)
plt.plot(x_values, y_values_2) # call plot again draws in the same graph.
plt.show()
```

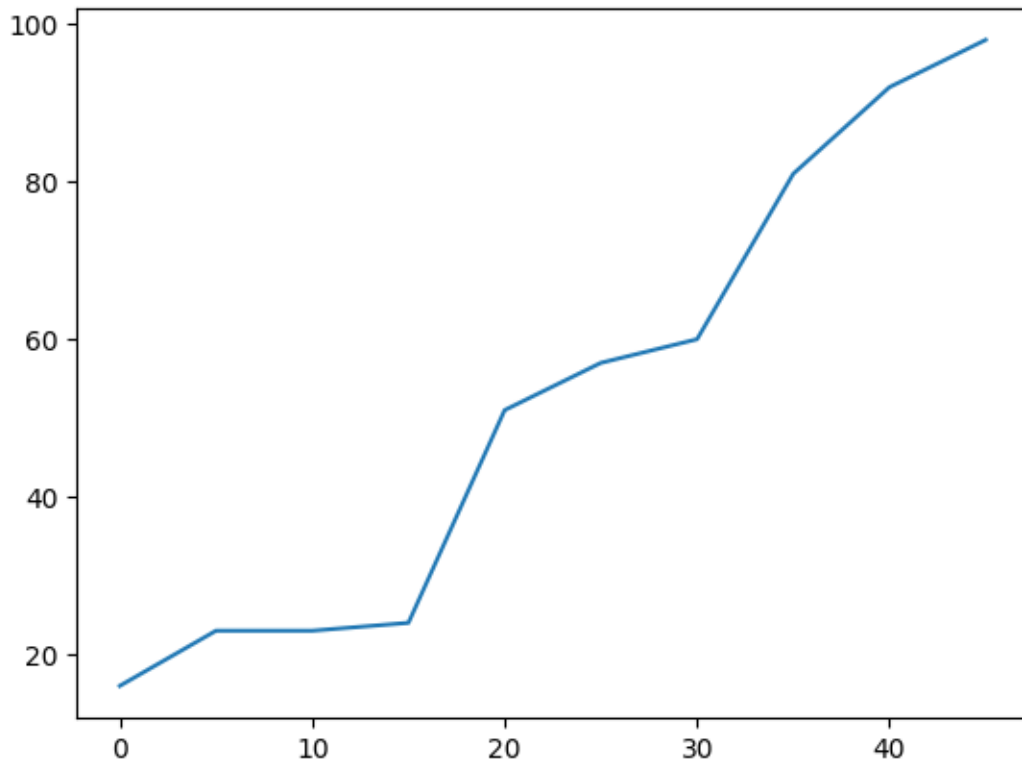


```
[8]: #
# Activity 1: Create data so that the plot draws an
# ascending line (y_values increase at any rate).
#
x_values = [ i for i in range(0,50,5)]
y_values = [random.randint(1,100) for i in range(0,10,1)]

print(x_values,y_values.sort())

plt.plot(x_values,y_values)
plt.show()
```

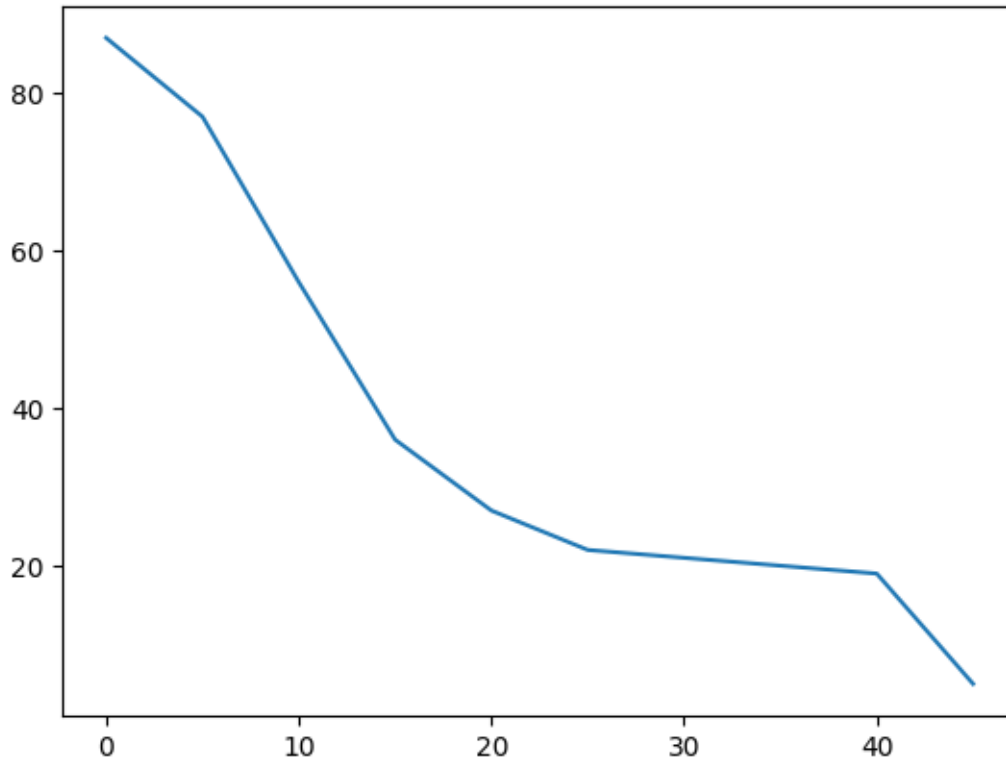
[0, 5, 10, 15, 20, 25, 30, 35, 40, 45] None



```
[13]: #
# Activity 2: Create data so that the plot draws a
# descending line (y_values decrease at any rate).
#
x_values = [ i for i in range(0,50,5)]
y_values = [random.randint(1,100) for i in range(0,10,1)]
y_values.sort(reverse=True)
print(f'X:{x_values}\nY:{y_values}')

plt.plot(x_values,y_values)
plt.show()
```

```
X:[0, 5, 10, 15, 20, 25, 30, 35, 40, 45]
Y:[87, 77, 56, 36, 27, 22, 21, 20, 19, 5]
```



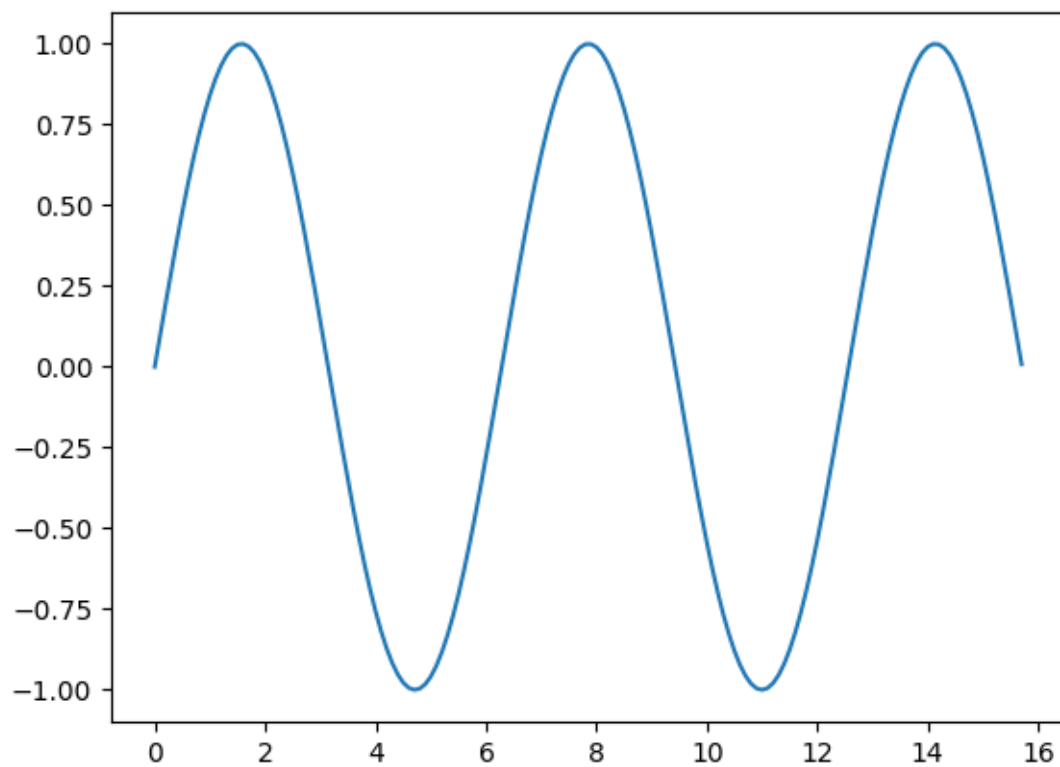
```
[36]: #
# Activity 3: Create data so that the plot draws a
# wave. You can consider using Python's math library, which has
# a sin function (detail https://www.w3schools.com/python/ref\_math\_sin.asp).
#
import math
import numpy as np
x_values = np.arange(0,5*np.pi,0.1)
y_values = np.sin(x_values)
#print(f'X:{x_values}\nY:{y_values}\nlen {len(x_values)}')
plt.plot(x_values,y_values)
plt.show()
```

```
X:[ 0.  0.1  0.2  0.3  0.4  0.5  0.6  0.7  0.8  0.9  1.  1.1  1.2  1.3
 1.4  1.5  1.6  1.7  1.8  1.9  2.  2.1  2.2  2.3  2.4  2.5  2.6  2.7
 2.8  2.9  3.  3.1  3.2  3.3  3.4  3.5  3.6  3.7  3.8  3.9  4.  4.1
 4.2  4.3  4.4  4.5  4.6  4.7  4.8  4.9  5.  5.1  5.2  5.3  5.4  5.5
 5.6  5.7  5.8  5.9  6.  6.1  6.2  6.3  6.4  6.5  6.6  6.7  6.8  6.9
 7.  7.1  7.2  7.3  7.4  7.5  7.6  7.7  7.8  7.9  8.  8.1  8.2  8.3
 8.4  8.5  8.6  8.7  8.8  8.9  9.  9.1  9.2  9.3  9.4  9.5  9.6  9.7
 9.8  9.9 10. 10.1 10.2 10.3 10.4 10.5 10.6 10.7 10.8 10.9 11. 11.1
11.2 11.3 11.4 11.5 11.6 11.7 11.8 11.9 12. 12.1 12.2 12.3 12.4 12.5]
```

```

12.6 12.7 12.8 12.9 13.  13.1 13.2 13.3 13.4 13.5 13.6 13.7 13.8 13.9
14.  14.1 14.2 14.3 14.4 14.5 14.6 14.7 14.8 14.9 15.  15.1 15.2 15.3
15.4 15.5 15.6 15.7]
Y: [ 0.          0.09983342  0.19866933  0.29552021  0.38941834  0.47942554
    0.56464247  0.64421769  0.71735609  0.78332691  0.84147098  0.89120736
    0.93203909  0.96355819  0.98544973  0.99749499  0.9995736  0.99166481
    0.97384763  0.94630009  0.90929743  0.86320937  0.8084964  0.74570521
    0.67546318  0.59847214  0.51550137  0.42737988  0.33498815  0.23924933
    0.14112001  0.04158066 -0.05837414 -0.15774569 -0.2555411  -0.35078323
   -0.44252044 -0.52983614 -0.61185789 -0.68776616 -0.7568025  -0.81827711
   -0.87157577 -0.91616594 -0.95160207 -0.97753012 -0.993691  -0.99992326
   -0.99616461 -0.98245261 -0.95892427 -0.92581468 -0.88345466 -0.83226744
   -0.77276449 -0.70554033 -0.63126664 -0.55068554 -0.46460218 -0.37387666
   -0.2794155  -0.1821625  -0.0830894  0.0168139  0.1165492  0.21511999
    0.31154136  0.40484992  0.49411335  0.57843976  0.6569866  0.72896904
    0.79366786  0.85043662  0.8987081  0.93799998  0.96791967  0.98816823
    0.99854335  0.99894134  0.98935825  0.96988981  0.94073056  0.90217183
    0.85459891  0.79848711  0.7343971  0.66296923  0.58491719  0.50102086
    0.41211849  0.31909836  0.22288991  0.12445442  0.02477543 -0.07515112
   -0.17432678 -0.27176063 -0.36647913 -0.45753589 -0.54402111 -0.62507065
   -0.69987469 -0.76768581 -0.82782647 -0.87969576 -0.92277542 -0.95663502
   -0.98093623 -0.99543625 -0.99999021 -0.99455259 -0.97917773 -0.95401925
   -0.91932853 -0.87545217 -0.82282859 -0.76198358 -0.69352508 -0.61813711
   -0.53657292 -0.44964746 -0.35822928 -0.26323179 -0.16560418 -0.0663219
    0.03362305  0.13323204  0.23150983  0.32747444  0.42016704  0.50866146
    0.59207351  0.66956976  0.74037589  0.80378443  0.85916181  0.90595474
    0.94369567  0.9720075  0.99060736  0.99930939  0.99802665  0.98677196
    0.96565778  0.93489506  0.89479117  0.84574683  0.78825207  0.72288135
    0.65028784  0.57119687  0.48639869  0.39674057  0.30311836  0.20646748
    0.10775365  0.00796318]
len 158

```



Module completion check

This is the final page in the module. In order to progress to the next module in this course, all of the assessment tasks listed below, need to be completed.

Assessment	Type	Completed
1.1 How to Complete the Compulsory Safety Induction	Visited topic	Complete
1.2 Introduction	Visited topic	Complete
1.3 What to Do and Not to Do	Grade item, Passmark: [100%]	Score: 100%
1.4 Respecting others	Grade item, Passmark: [100%]	Score: 100%
1.5a Equipment Use	Grade item, Passmark: [100%]	Score: 100%
1.5b Equipment Use	Grade item, Passmark: [100%]	Score: 100%
1.6a Emergency	Grade item, Passmark: [100%]	Score: 100%
1.6b Emergency	Grade item, Passmark: [100%]	Score: 100%
1.7 General Online Induction Quiz	Grade item, Passmark: [80%]	Score: 93%
1.8 General MakerSpace Induction (Online Module) Summary Page	Visited topic	Complete

Congratulations, you've successfully completed the General Online Induction Module.

You can now click the 'Collect Your Certificate' button below to collect your certificate of complete.

Please note: You'll now need to book a time to visit the Makerspace in person so that one of our friendly mentors can do the In-Person Check

Please follow the link to book your [in-person general induction](#).

[< Previous](#)

[Collect your
Certificate](#)

Module completion check

This is the final page in the module. In order to progress to the next module in this course, all of the assessment tasks listed below, need to be completed.

Assessment	Type	Completed
6.1 Safety Induction for Soldering	Visited topic	Complete
6.2 Soldering Introduction	Visited topic	Complete
6.3 Workflow: Theory Behind Soldering	Visited topic	Complete
6.4 Workflow: Soldering in Makerspace	Visited topic	Complete
6.5 Workflow: Things to Consider When Soldering	Visited topic	Complete
Soldering Induction Assessment Quiz	Grade item, Passmark: [80%]	Score: 100%

Congratulations, you've successfully completed the Soldering Online Quiz and Module.

You can now click the 'Collect Your Certificate' button below.

Please note: You'll now need to follow this link to book an **in-person Soldering Safety Induction**. Passing the soldering in person check is a requirement for being able to use the soldering stations unsupervised.

[< Previous](#)[Collect your Certificate](#)